Digital Clock

Yashraj Kranthi

17,August 2026

1 Collection of Components

Collected the components from core labs on 17th August,2026

2 Testing of Components

2.1 Testing of Arduino



Figure 1: Ran a basic blinking program through Arduino IDE to verify that the Arduino was working correctly

The test was successful, confirming that the Arduino was working properly.

I learned the use of basic functions like digitalWrite, pinMode, digitalRead, setup(), loop(), and delay().

2.2 Testing of Seven Segment Display

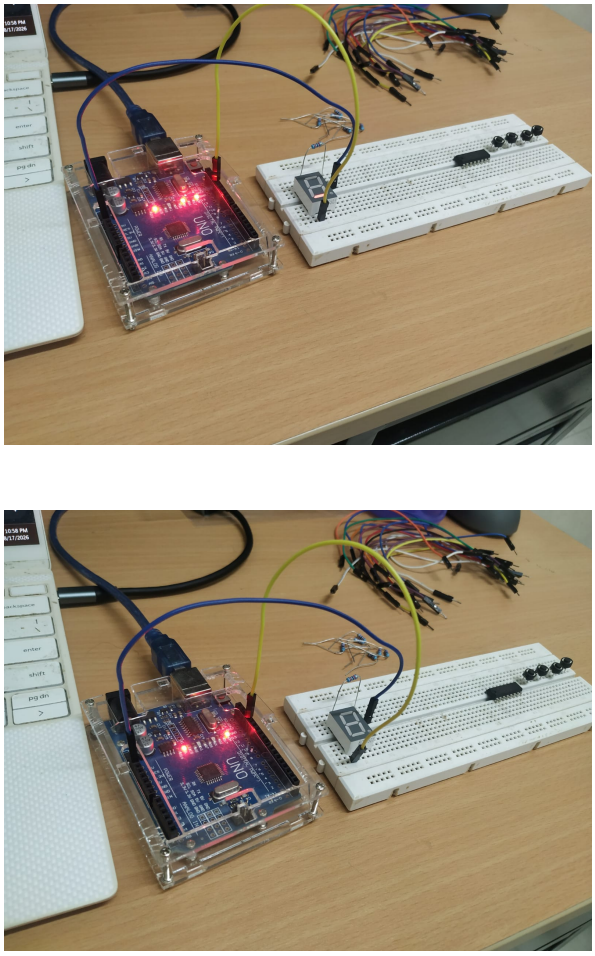


Figure 2: One of the LEDs of the seven segment display was not working during testing

This made me realise that it is better to check the components individually before placing them in the final circuit, as otherwise it would be difficult to tell whether an error came from the display or the wiring.

2.3 Testing of IC 7447

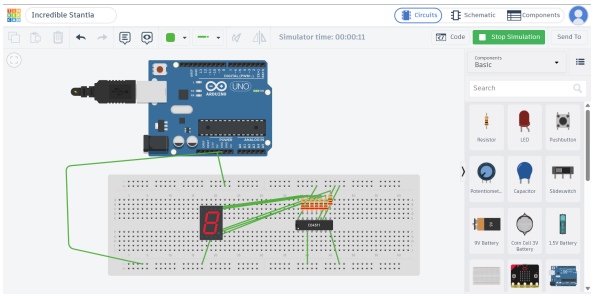


Figure 3: Tried an initial circuit for testing the IC 7447 through Tinkercad

I tried making an online circuit and I used the CD4511 only as a substitute for simulation because the 7447 was not available in Tinkercad. I did not treat it as an exact replacement for the physical 7447. The CD4511 works similarly to the IC 7447, but it is designed for common cathode displays, whereas the 7447 is used with common anode displays. So, after connecting common to ground instead of 5V, it worked as normal. I initially forgot to add the 220 ohm resistors, which caused the LED to be connected directly to the output instead of having current limited. This led to both seven segment display and the CD4511 to get damaged in the simulation.

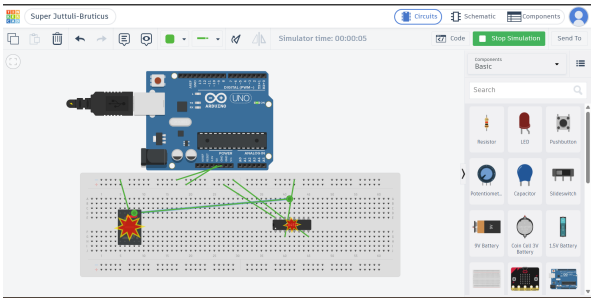


Figure 4: Damaged display and CD4511

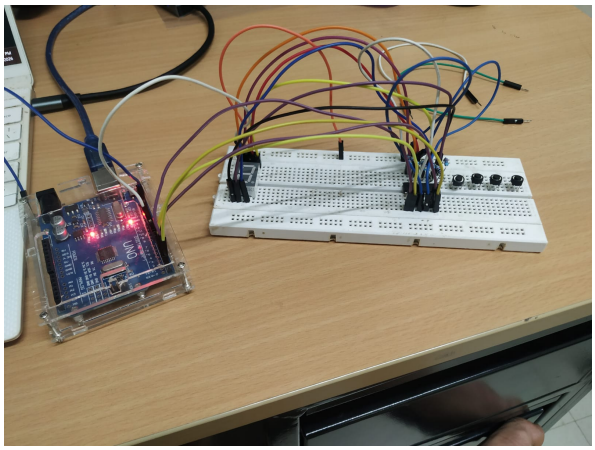


Figure 5: Testing the IC 7447 by connecting its outputs to a single seven segment display

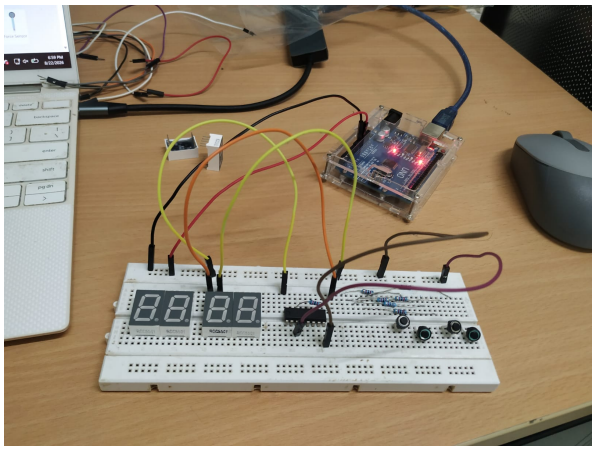


Figure 6: The IC 7447 was still not producing the expected display during testing

I thought I was not getting the required display because I had connected some of the wires incorrectly, so I rewired it to make a simpler test circuit, IC 7447 has both 5V and ground connections, the g segment LED connection on the seven segment display with the g pin(pin 14) on the IC 7447 through a 240 ohm resistor, I connect pin 3 on the IC 7447 to ground. I tried the same connection online, where the g segment on the display lit up, but my actual circuit was not lighting up. By this point I could not yet determine whether the problem was the IC, the display or my wiring, so I decided to isolate the components more and test them out individually.

At this point, I suspected that the IC 7447 might be faulty because the display worked normally when connected directly through a resistor, and I had checked the connections

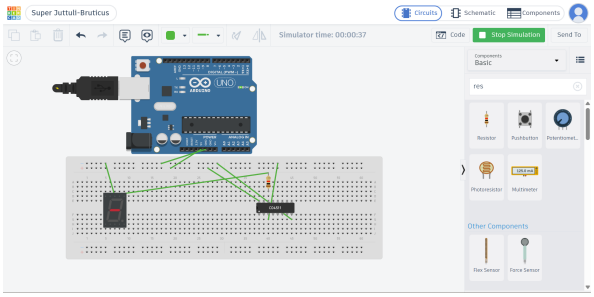


Figure 7: Online circuit simulation of the IC 7447 test

I went to the core labs again and got another IC 7447 but after returning and checking it again, I realized that even the IC 7447 I got again was not working

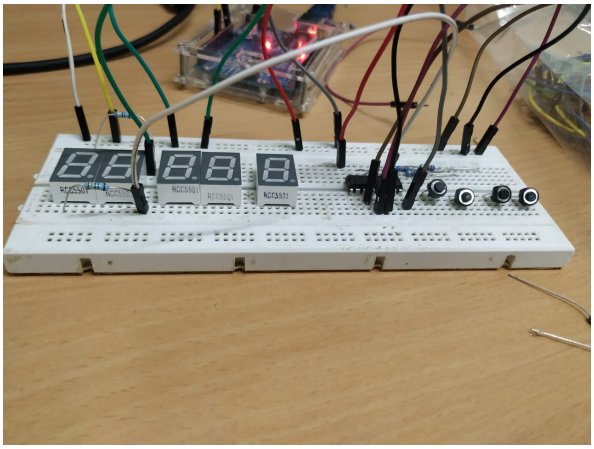


Figure 8: The basic IC 7447 connection was still not producing the expected output

2.4 Testing the Buttons

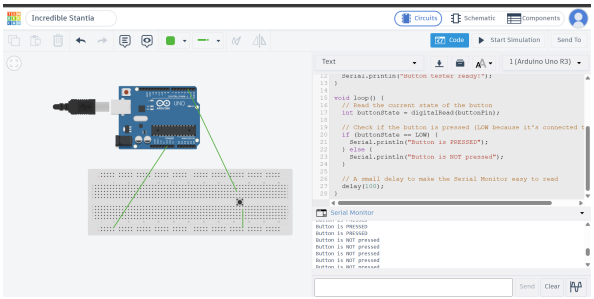


Figure 9: Experimented with different button connections in an online circuit to understand how the buttons worked

Originally, I forgot about the resistors and the input was sometimes reading HIGH and sometimes LOW even without a button press. I realized that the input was floating because it did not have a defined default state. I learned that a pull-up/pull-down resistor gives the input a defined state when the button is not pressed.

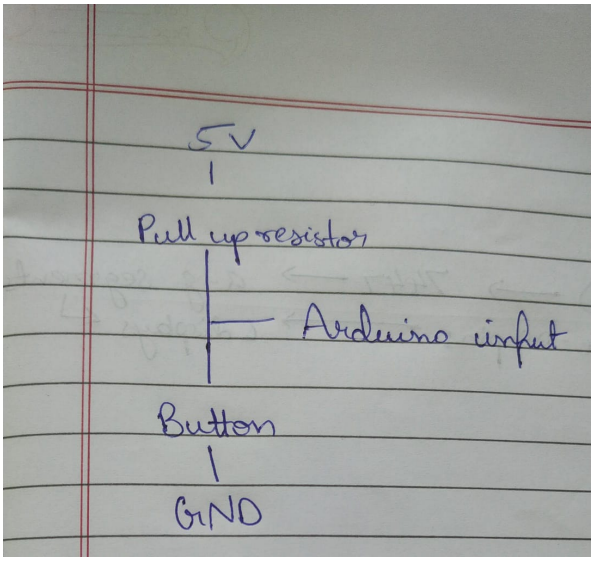


Figure 10: Diagram explaining how a pull-up resistor works

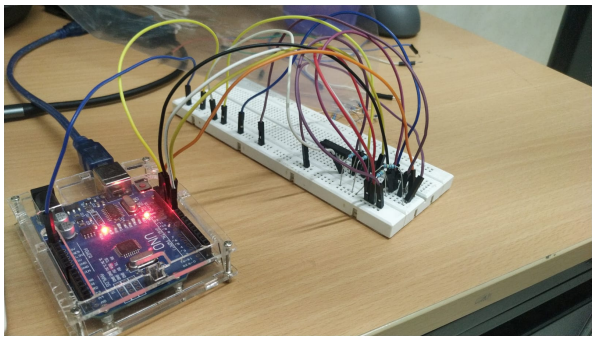


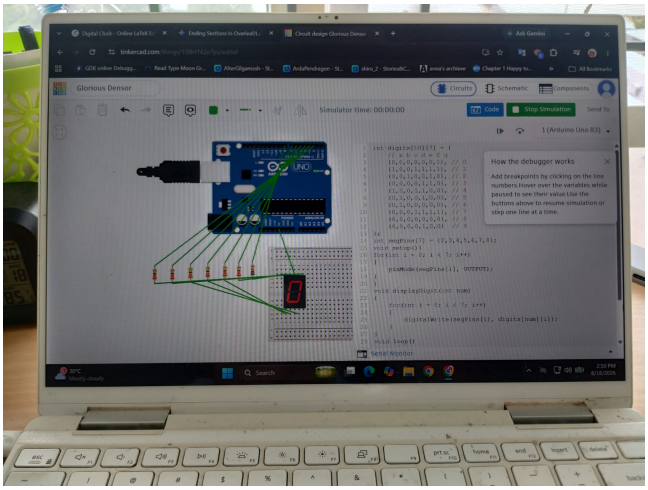
Figure 11: Uploaded a basic Arduino program to test the buttons using the Arduino IDE Serial Monitor

I checked the Serial Monitor while pressing each button and verified that the Arduino detected the button presses correctly.

This also showed me that when debugging a hardware project, I should change one part at a time so that I can identify what caused the problem.

3 Running basic codes through Arduino

3.1 Running 0-9 digits on Arduino through Tinkercad(18th August, 2026)



The display was showing random digits and some segments were not lighting. I suspected either incorrect wiring or a faulty connection. I rewired the connections and the display began counting correctly, which indicated that the problem was caused by the wiring rather than the code.

4 Wiring the circuit

4.1 Initial wiring of circuit of Tinkercad

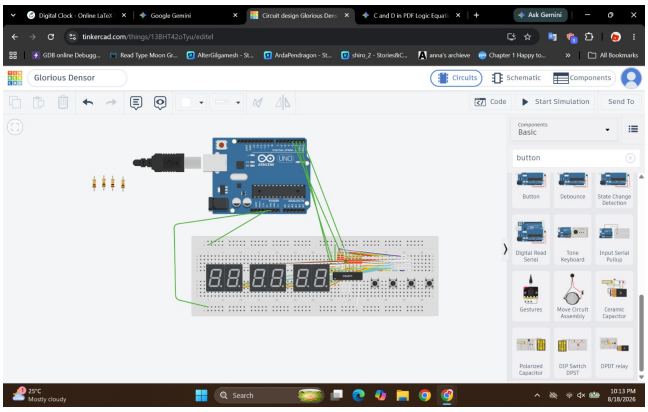


Figure 12: Initial Tinkercad circuit connecting the IC 7447 to the Arduino and seven-segment display through 220 ohm resistors

4.2 Initial wiring of circuit(18th August,2026)

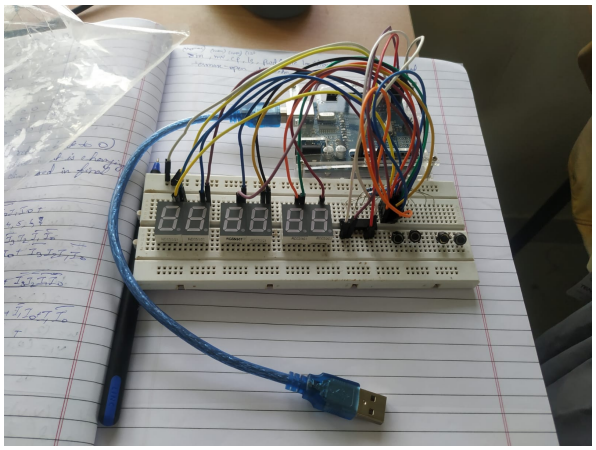


Figure 13: Initial physical wiring was stopped because of a damaged seven segment display and not having enough jumper wires

I decided that continuing with the same layout would make later debugging and wiring more difficult, so I removed the components and planned to reorganize the circuit.

5 What I Learned Today(18th August,2026)

1. C language fundamentals (Source - CS50x Week 1)

   Revised basics like variables, loops, if/else statements and functions

2. Arduino Fundamentals (Source - Paul McWhorter(YT))

   I learned how setup() and loop() work and how basic digital input/output is handled in Arduino

6 Creation of circuit online

6.1 Finished wiring the circuit online

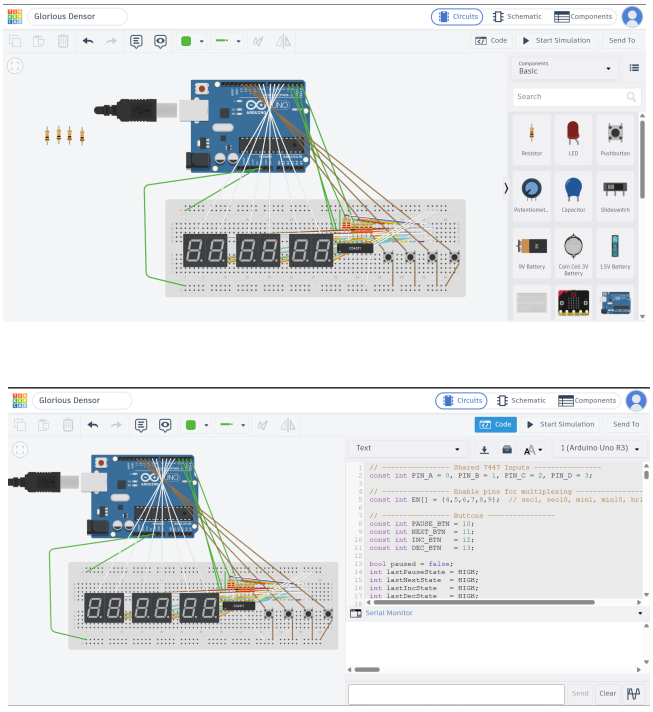


Figure 14: Uploaded the clock code to test the online circuit

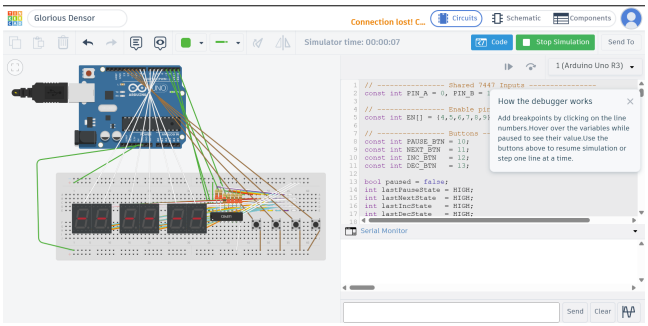


Figure 15: The online circuit was not working as expected when the clock code was run

I encountered a problem when I ran the simulation for the circuit, I used the CD4511 component in place of the IC 7447 when I couldn't find the IC 7447 in Tinkercad. While running the simulation I realized that the CD4511 works for common cathode seven segment display and not for the common anode seven segment display.

This helped me understand that the decoder cannot be treated as independent of the type of seven-segment display being used, the decoder and display configuration have to match.

7 What I Learned Today(19th August,2026)

Source Neso Academy(YT)

1. Binary I learned how numbers are represented using only 0s and 1s and how binary values correspond to decimal numbers. This helped me understand how digital circuits represent and process information

2. Number Systems I learned about different number systems such as decimal and binary, and how the same value can be represented in different ways. This made it easier to understand the different forms of numbers used in digital electronics

3. BCD I learned that in Binary Coded Decimal, each decimal digit is represented separately using 4 bits. This is important for the clock because the Arduino sends the digits to the 7447 in BCD form

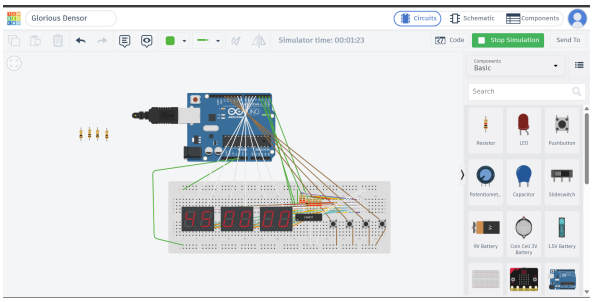
4. Logic Gates (Recap) I revised the basic logic gates such as AND, OR, NOT, XOR and XNOR and how they produce outputs depending on their inputs. This helped me understand the logic behind the digital circuits used in the project

5. Boolean Algebra (Recap) I revised how Boolean expressions can be used to represent logic and how different combinations of AND, OR and NOT

operations can describe the behavior of a circuit. This will help me understand the Boolean equations and K-maps used later in the clock design

8 Attempted to create a working online circuit

8.1 Trial 1



I tried to adjust the code so that it will work for CD4511 instead of IC 7447,I changed the logic for the fact that the LEDs will turn on when the Arduino pin gives LOW output. I also changed the seven segment displays from common anode to common cathode and grounded the pin 5 of CD4511 instead of supplying 5V to it as in the case of IC 7447.

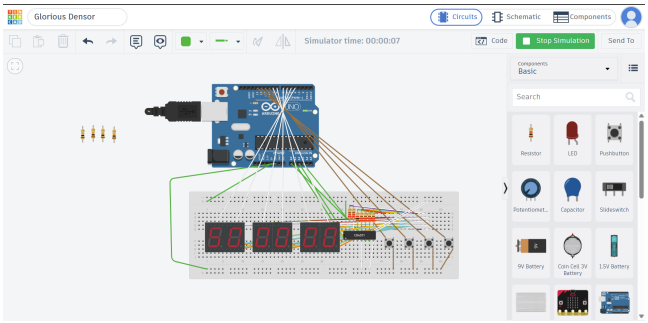


Figure 16: Result of Trial 1 after modifying the code for the CD4511 and common cathode display

The clock was not working properly as the hour display was increasing every second like the seconds display should.

8.2 Trial 2

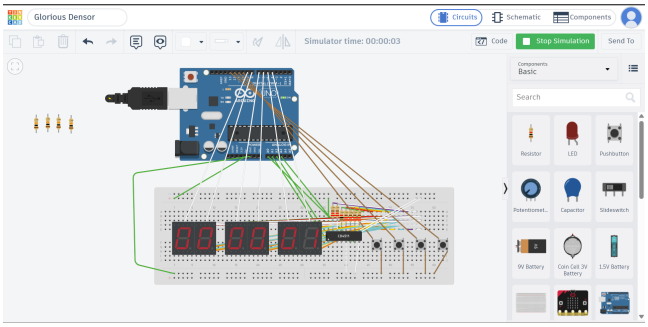


Figure 17: Result after rewiring the circuit in Trial 2

The clock is working normally, though the clock is not counting correctly. 1 second on the online clock corresponds to about 3 seconds. When measured alongside the simulation time, it matches with the simulation time. It looks like the online simulation was not able to perform all the necessary calculations fast enough and hence the delay. I originally saw the way the displays were updating slower than expected and thought I made another mistake.

9 What I learned (20th+21st August,2026)

1. Workings of seven segment display decoders(IC 7447 and CD4511)

   The decoders receive a BCD(binary coded decimal) input through the pins 7,1,2,6 which are Least Significant to Most Significant respectively. The decoders then provide a LOW output for the segments which are to be turned on through the respective pins on the decoders. The pins 3,4,5 have functions LT(turns all all segments), BI(turns off all segments), RBI(Turns off leading zero)

2. Basics of Multiplexing

   Learned how a digital clock uses multiplexing to save pins, the Arduino sends the BCD value for one digit and enables that digit, then rapidly moves to the next digit

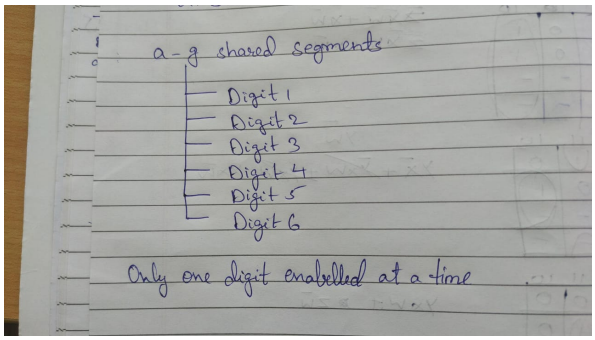


Figure 18: Basic idea of multiplexing in the clock. The seven segment lines are shared between the displays, while the Arduino enables one digit at a time

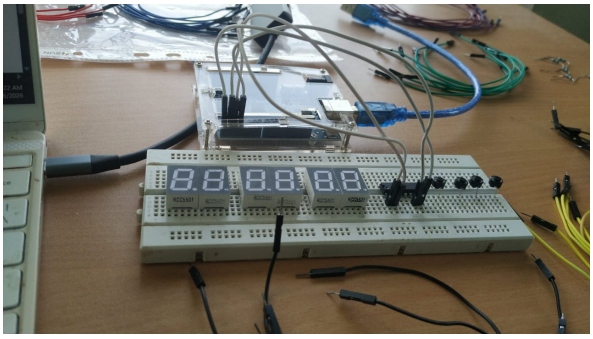
3. How does the Arduino display the clock?
4. How do pull up and pull down resistors work

   The resistor acts as a weak bungee cord, holding the Arduino pin at a default voltage when the switch is open. Pressing the switch creates a direct connection, a strong, solid cable that easily overpowers the bungee cord and dictates the pin's new voltage
5. Functioning of buttons

   Learned how pull up and pull down resistors are necessary for the working of the button.

10 Wiring the physical circuit

10.1 Placing major components on the breadboard



I connected the seven segments displays, the IC 7447, the buttons and also connected the ABCD inputs (pins 7,1,2,6) on the IC 7447 to the respective Arduino pins (D0,D1,D2,D3).

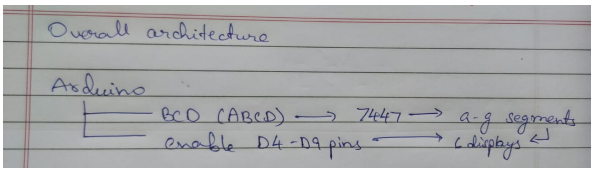


Figure 19: Basic layout of how the Arduino, 7447 and six display digits are connected in the clock

10.2 Self realization of my error

I was tinkering with some resistors and I realized that all this time I was using 10K ohm resistors in the place of 220 ohm resistors, I realized this was the reason my seven segment displays were always so dim when turned on. I found out that the ones I was using were 10k ohm ones when I took new 240 ohm resistors because I cut the resistors I was working with too short and when I was testing it with the IC 7447, Seven Segment Display, the display was very bright in comparison with my previous experiences and later I searched online as to what the color coding for 10k ohm resistors and realized I was using 10k ohm resistors. Initially I got confused with 240 ohm and 10k ohm resistors and thought 10k ohm resistors were 240 ohm resistors. I also asked the person in charge of the equipment distribution and learned that there were no 220 ohm resistors and he was giving 240 ohm resistors in their place.

I also realized that the IC 7447 which I got from core labs did not work as well when I tried the basic test for IC 7447. I tried connecting pin 16 to 5v, pin 8 to gnd, pin 3 to gnd and one wire from the resistor connected to pin 15 of IC 7447 to segment f of a seven segment display.

I thought that the resistor may not have been working and also changed the previous 10k ohm resistor with a new 240 ohm resistor and it still did not work. At this point, I concluded that the IC 7447 was probably defective. It was late into the night, so I decided to just wire the circuit without the presence of the IC 7447 and to just add it after getting it again tomorrow.

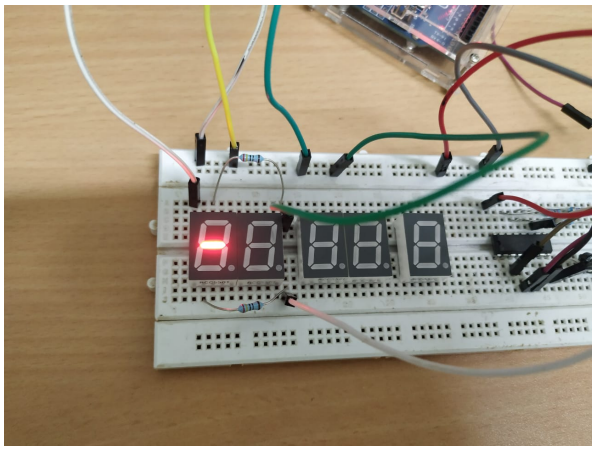


Figure 20: Seven segment display with the correct resistor value, producing a brighter output

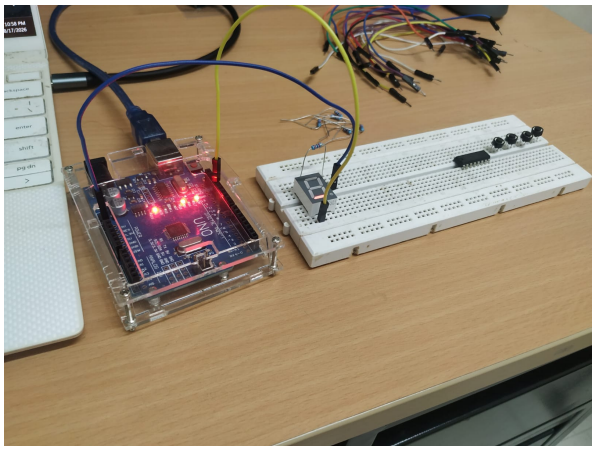


Figure 21: Seven segment display with a dim output caused by my earlier resistor value mistake

The resistance of the 10k ohm resistor was much higher than the 240 ohm resistor, so the current through the LED was much lower and the display appeared dimmer.

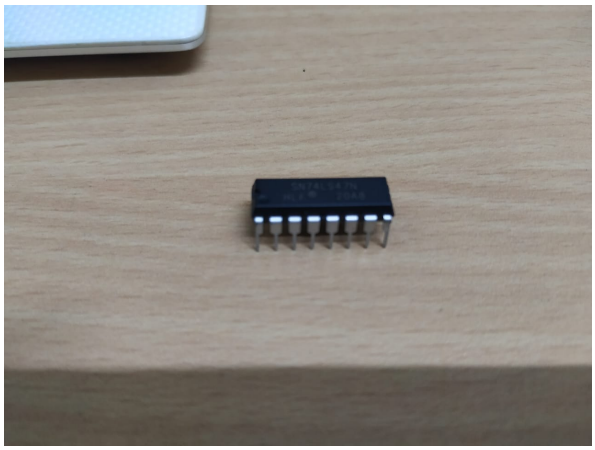


Figure 22: IC 7447 that I suspected was defective after the initial testing

10.3 Trying out different colour codes

I was trying to group all similar functioning jumper wires to have the same colors.

I wanted the wiring to be easier to follow and to make it easier to identify similar connections while debugging.

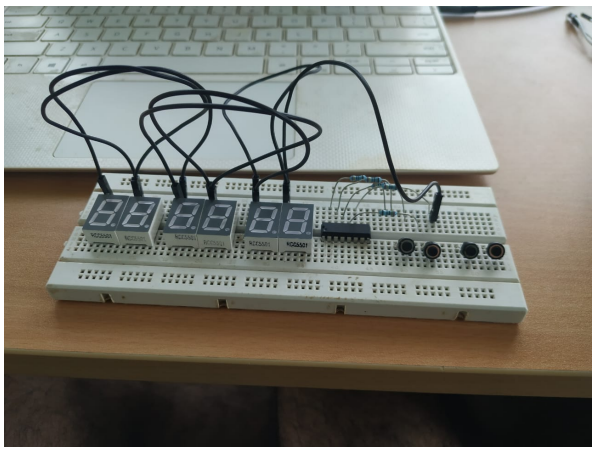


Figure 23: Grouping the black jumper wires together to see how the wiring would look

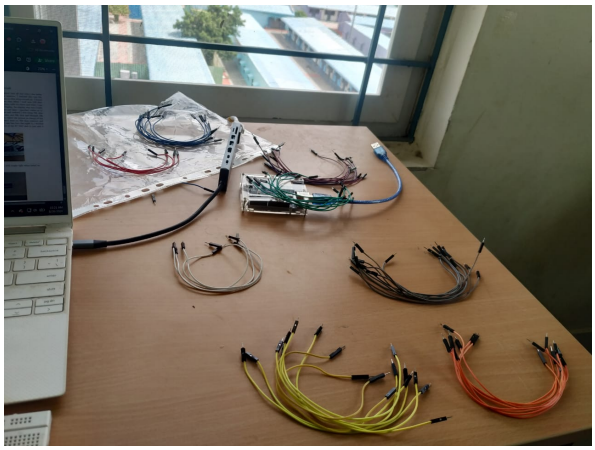
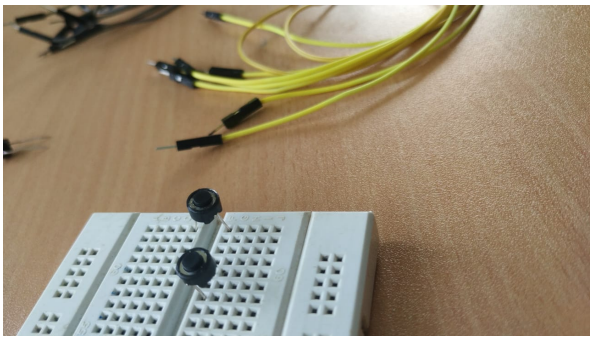
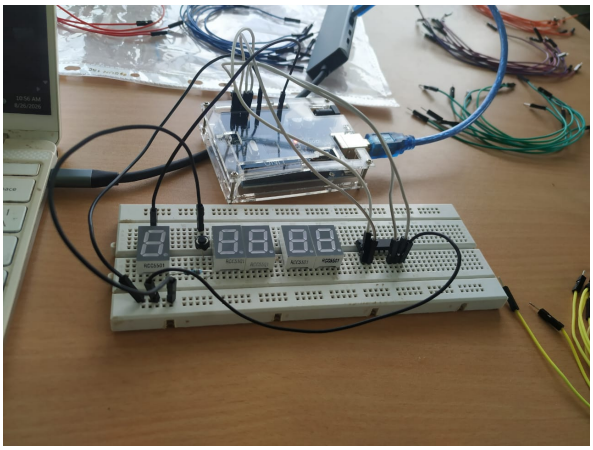


Figure 24: I grouped all the jumper wires I have by the colour(took a lot of time)

10.4 Deciding which way to connect the buttons



I decided to use the second type(the one where the center is between the legs of the button)



I decided to re-test my buttons as well,I connected the button so that pressing it would complete the path and light the decimal-point LED. I repeated the test for all four buttons, and all of them worked.

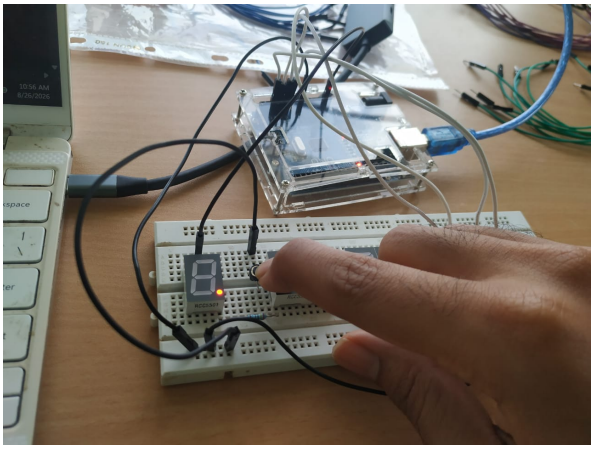
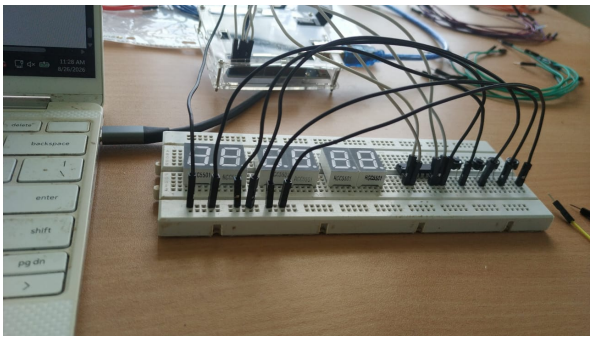


Figure 25: Button test was successful for all four buttons

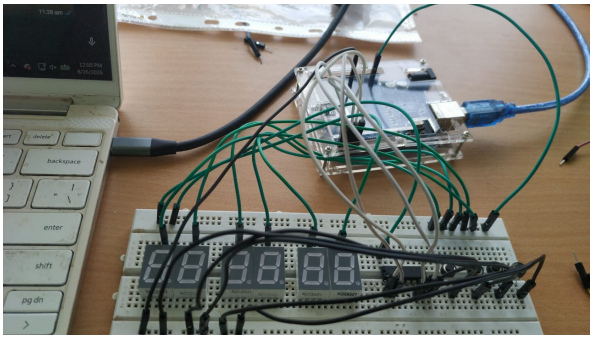
10.5 Finished wiring all the ground connections with black wires

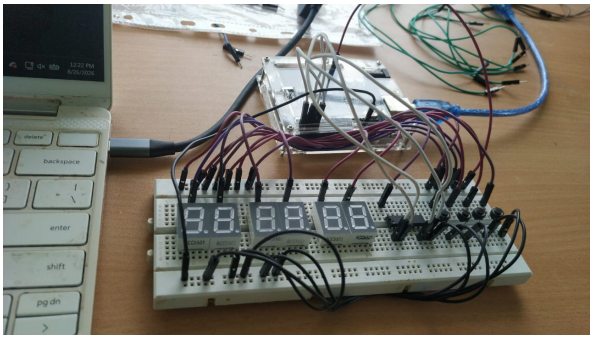


10.6 Tried to wire all 5v connections with green jumper cables

There were not enough green wires for all the necessary connections.

I kept the same colour for all the 5V connections so that they would still be easy to identify even though it was not the colour I initially planned.





I wired all 5V connections using violet jumper cables.

10.7 Made a calculation error

I thought there was enough space and I left a lot of space between components but I forgot about the resistors and the connections to respective segments of seven segment displays. Now I have to take apart everything and start over again.

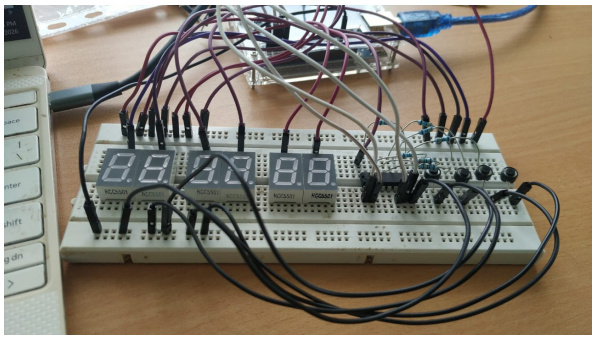


Figure 26: Original wiring layout before I realised there was not enough space for the resistors and segment connections

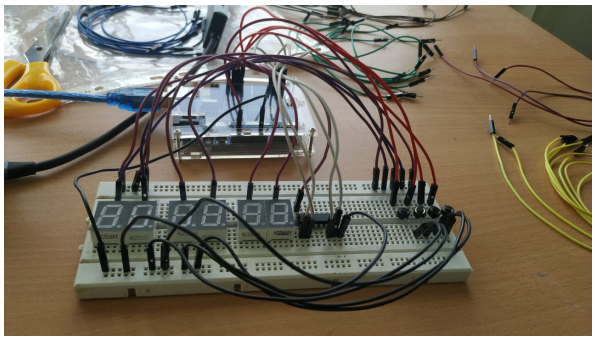


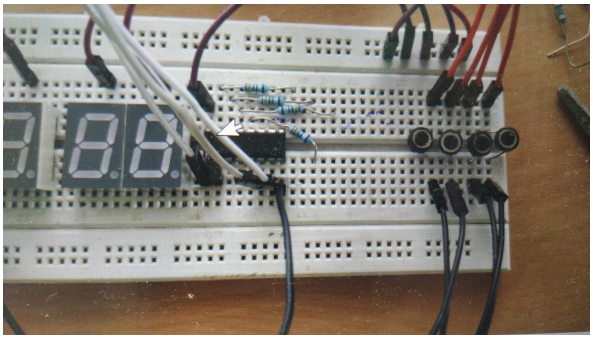
Figure 27: New wiring layout after rearranging the components to make space for the resistors and segment connections

This taught me that I should account for the space occupied by the connections and resistors, not just the main components, when planning a breadboard layout.

10.8 Resistor placement problem

I tried cutting the resistors to shorten them but along the way I realized that having resistors touching the breadboard means that they will cover the bread-

board space where I have to connect the wires to different segments of the seven segment display. This also increases the chances of errors when running the clock as the metal parts of the resistors may touch and cause errors.



Later I decided to wire some resistors diagonally from the top half of the breadboard to the bottom half.

10.9 Re-wiring because of overlapping connections

I wired some of the violet wires above the white wires and some of them below the white wires, hence I was not able to effectively tape the wires and hence had to re-wire the violet and white wires.

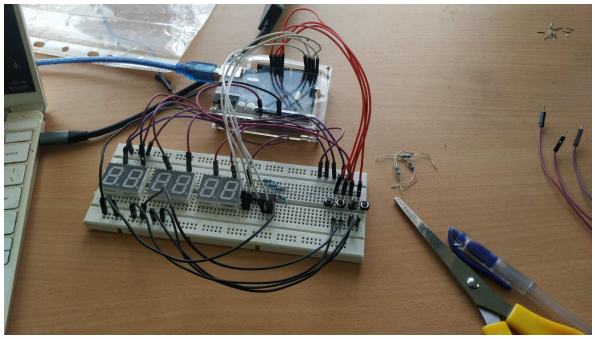


Figure 28: Wiring after rearranging the violet and white jumper wires

10.10 Testing the replacement IC 7447

I got another IC 7447 and wired it again but the seven segment display was still not working. I also tried to connect it to a separate seven segment display and tried to connect com to 5V and put a resistor to dp pin and connected it to gnd, that too did not work. I tried again by using different wires and finally got the display to work.

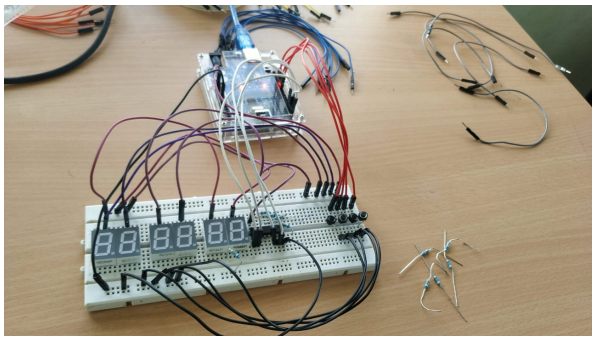


Figure 29: Seven segment display not turning on during the IC 7447 test

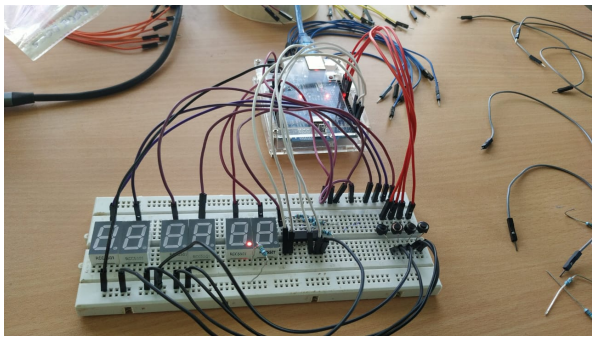


Figure 30: Seven segment LED lighting successfully after changing the connection

I realized that the problem was in the breadboard, or rather was in my understanding as I had assumed that the upper and lower rails continued electrically across the middle, but they were separated. This explained why some of my connections appeared to be correct physically but were not electrically connected.

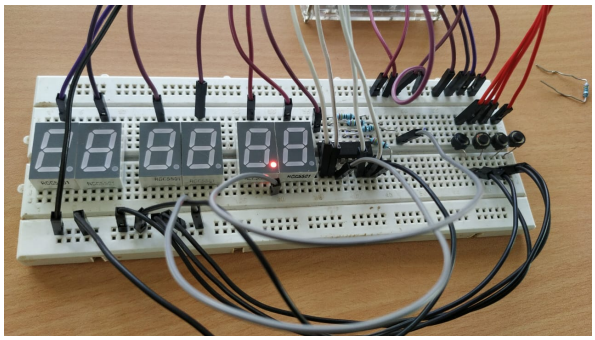


Figure 31: IC 7447 working after correcting the breadboard connection

The IC 7447 is also working when I connected pin 3 to gnd and pin 16 to 5V, pin 8 to gnd and connected the a pin to the dp pin of seven segment display through a 240 ohm resistor.

This test ensured that there were no other errors because of the IC 7447 and the display.

10.11 Connecting the resistors

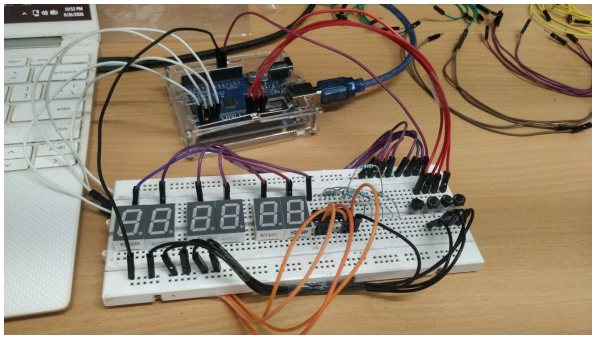


Figure 32: Initial placement of the resistors for the seven segment connections

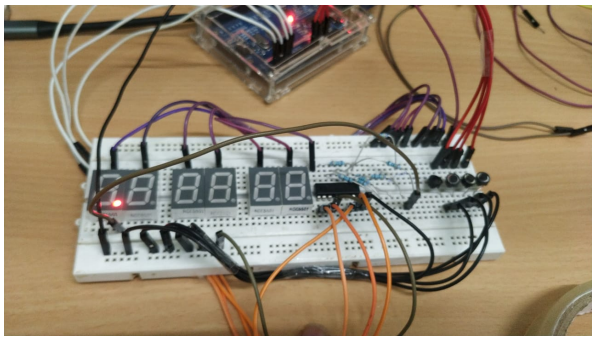
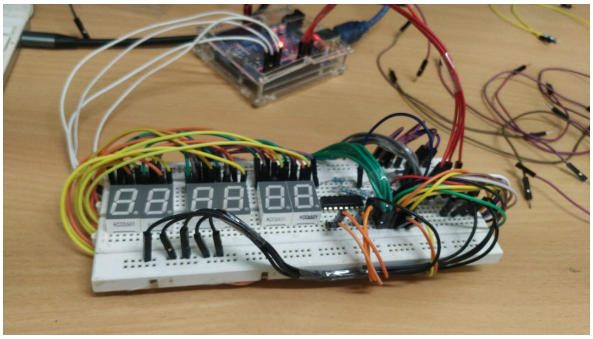
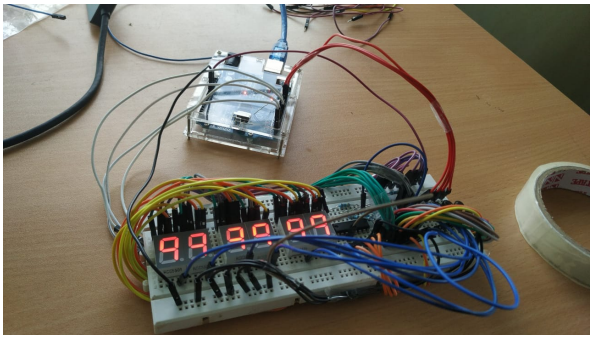


Figure 33: Checking the resistor lead lengths after cutting them

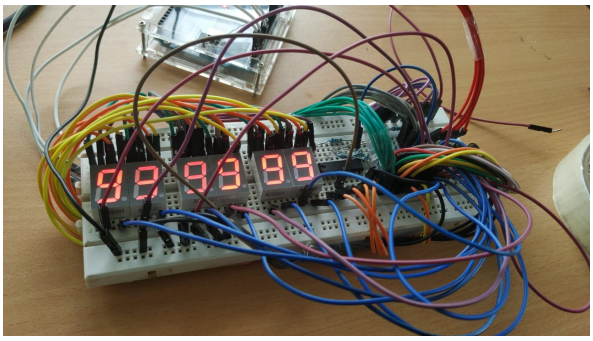
10.12 Finished connecting till b segment of all seven segment displays



10.13 Finished connecting till c segment of all seven segment displays



10.14 Finished connecting till d segment of all seven segment displays



10.15 e segment connections are not working

I connected e segment connections to two seven segment displays and turned the circuit on but the e segments on the respective seven segment displays were not working, I am trying to remove the resistor and place it again to see if there is any connection issue.

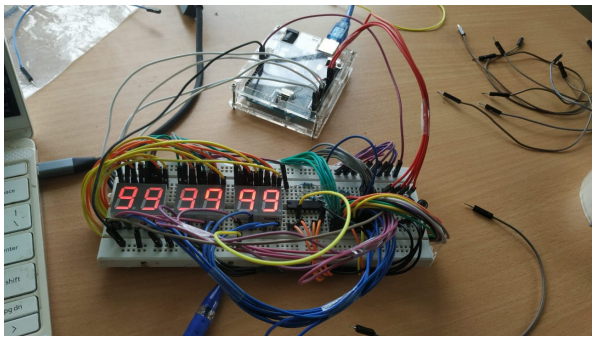


Figure 34: The e segments on two displays were not lighting up during testing

I removed the resistor but placing it again with all the wires was hard but I managed to place it with my scissors and the LED lit up hence proving that it was only a resistor connectivity issue.

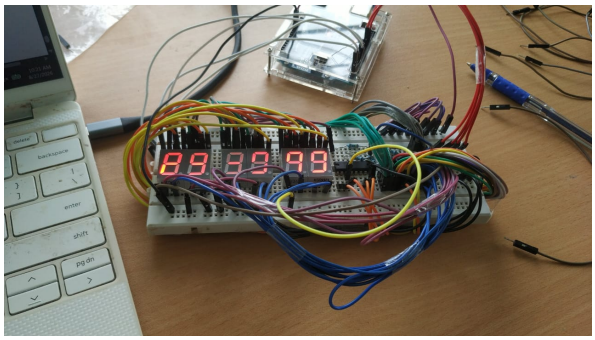
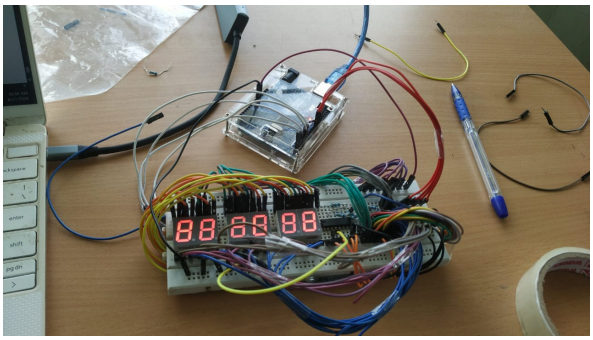


Figure 35: e segments working after reconnecting the resistor

10.16 Finished wiring all segments

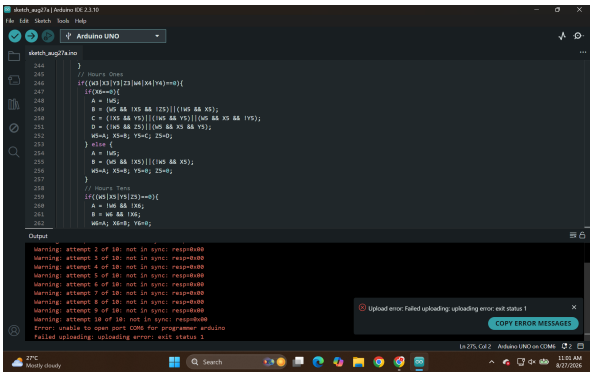
I wired all the pins of the 7447 to the respective segments of the seven segment displays.



11 Running the code

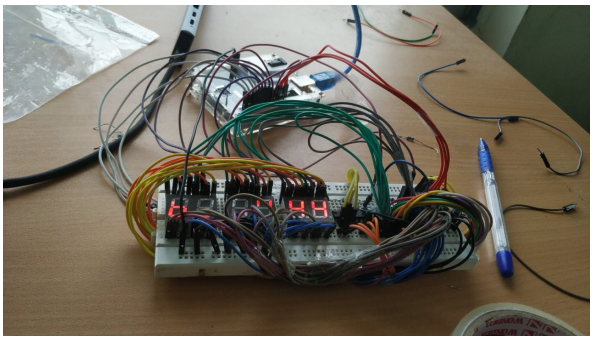
11.1 Trial 1 — Initial upload

I tried uploading the given code, but the Arduino IDE showed an error.



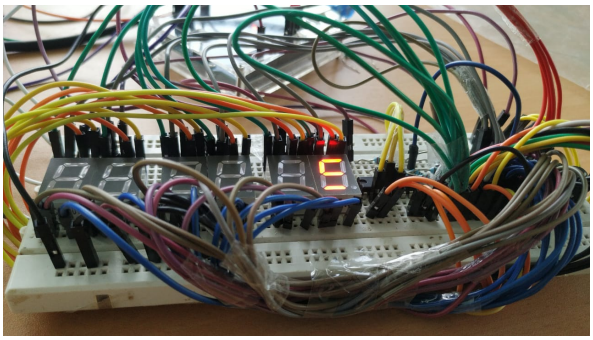
11.2 Trial 2 — Changed BCD connections

I moved the D0-D3 jumper cables to A0-A3 pins on the analog section of Arduino and re-ran the code. It was not working as the displays were only displaying 4-7 and then repeating.



11.3 Trial 3 - 4-7 problem

I removed the connections to all the displays except the seconds display and ran a basic code to count from 0-9 and then to rollover. The display was still counting from 4-7 and then rolling over. I checked the pins for ABCD, pin 16, pin 8 and still don't know about what is going wrong.



I later realized that I had connected a set of jumper wires to High(5V) and connected them to pins 2,3,4 instead of pins 3,4,5.

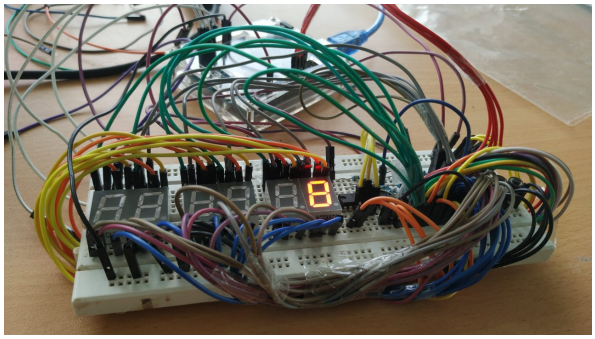


Figure 36: Display working correctly after correcting the BCD connections

This showed that a single incorrect fixed HIGH connection can completely change the BCD input being given to the decoder, which in turn changes the displayed digits.

11.4 Trial 4

I connected all the wires and uploaded the code, the clock is now working but there is a problem with some loose wiring particularly with the gray f segment wires, they seem a bit loose and affect the f segment of all seven segment displays.

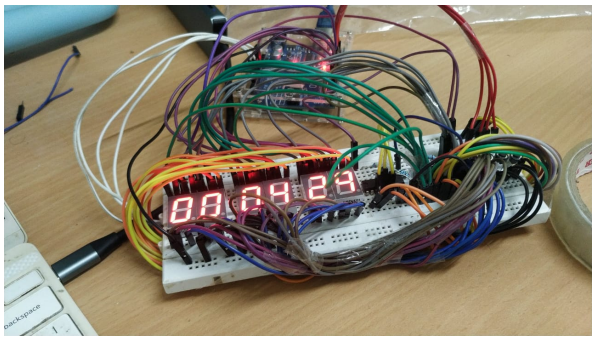


Figure 37: Full clock working after connecting all the displays

This made me realize that physical reliability of the circuit is just as important as getting the logic right.

12 Final clock body

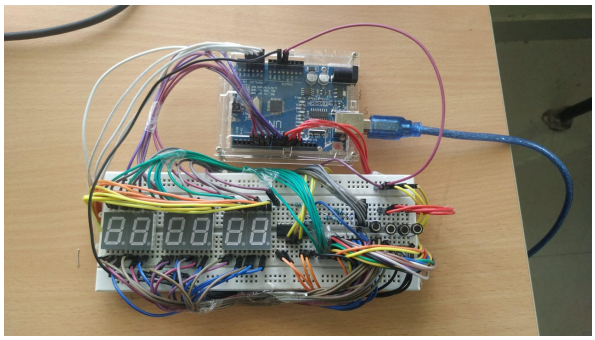


Figure 38: Final clock body when displays are turned off

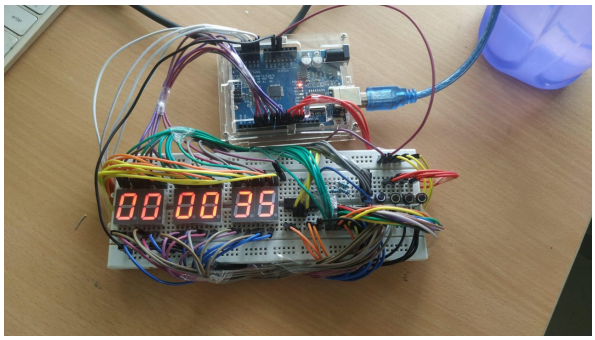


Figure 39: Final clock body when displays are turned on

13 Testing the clock

13.1 The Clock's Internal Timer Continues Running During Edit Mode

- Problem

  The displayed time jumped forward after leaving edit mode and did not update to the time set upon exiting the toggle mode

- Observation

  The system considered the total elapsed time even while in toggle mode, and it failed to update to the time left when exiting that mode

- Cause identified

  millis() continued to run while in edit mode.

- Change made

  Updated the pause/edit logic

- Result

  The manual time adjustments are successfully retained upon exiting toggle mode, and the clock resumes counting from the user-edited time without factoring in the background time spent editing.

I was running the clock and testing its features when I was in toggle mode, tried all the buttons, changed different digits and when I exited edit mode and came back to normal mode, it did not start off from where I exited the edit mode but instead started off from what the time would have been if it did not enter edit mode.

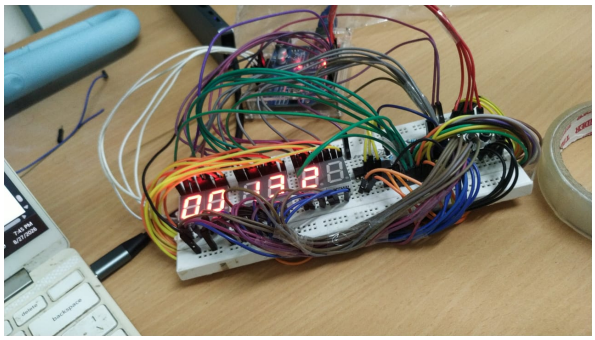


Figure 40: Entered edit mode at 00:13:22

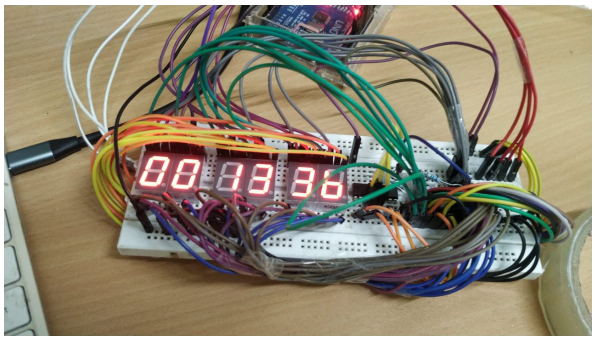


Figure 41: Clock resumed from 00:13:36 after exiting edit mode

The clock gave an increment of 14 seconds even when I hadn't changed any digits and was only in toggle mode . Another similar issue is in the case where even if I modified the time to change by a few hours, the time did not change according to what I edited in toggle mode but gave the time what had elapsed from when the clock stated, not factoring in the time I spent in toggle mode, editing the digits.

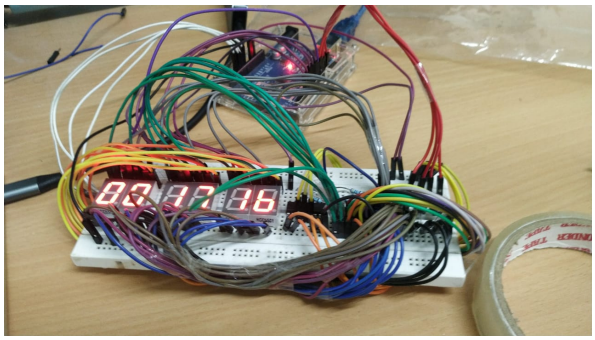


Figure 42: Clock entering edit mode at 00:17:16

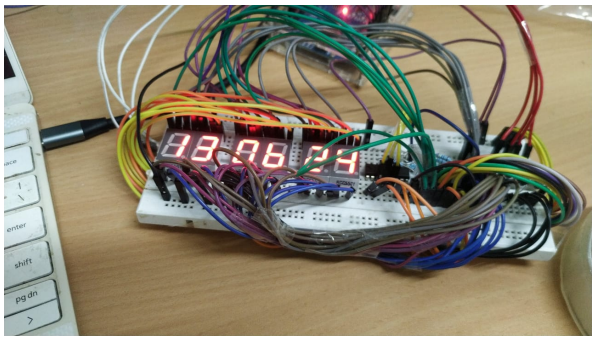


Figure 43: Changed the clock value to 13:06:24 while in edit mode

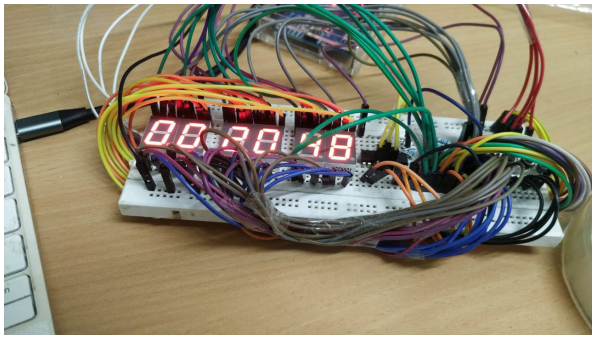


Figure 44: Clock resumed at 00:20:48 instead of the edited value of 13:06:24 after leaving edit mode

There was an error in the pause section of the code, when I paused the clock it is true that it entered toggle mode but the millis() function was still counting

the time even when in toggle mode and when it exited the toggle mode, it would satisfy the updating criteria and would update according to millis().

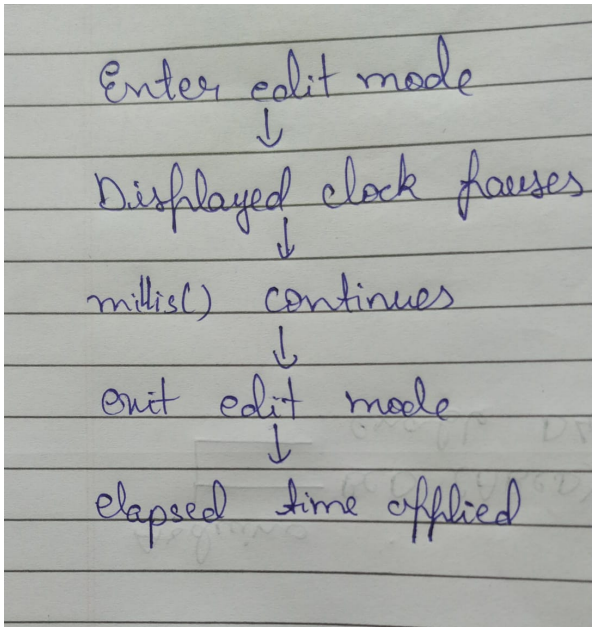


Figure 45: Simplified explanation of the edit-mode timer problem: the displayed clock was paused, but millis() continued advancing in the background



Figure 46: Original code containing the pause/edit-mode logic before the change



Figure 47: Updated pause/edit-mode logic after identifying the timer issue

13.2 Testing what happens when display power is disconnected

I got curious about what would happen to the displays if I suddenly disconnected the power, would the timer just reset?

First, I tried unplugging the 5V wire going from the Arduino to the breadboard. When I plugged it back in, the time displayed on the screen was not the same as when I disconnected it. Next, I tried unplugging the main Arduino power cable (the USB connected to my laptop). When I plugged that one back in, the timer completely reset and started over from 00:00:00.

It took me some time to figure it out, but the explanation is actually really easy. When I only disconnected the 5V wire to the breadboard, the Arduino itself was still powered by my laptop. It was running the timer code and doing all the calculations in the background the whole time; I just couldn't see the numbers ticking up because I had cut the power to the displays.

13.3 Button behavior with display power removed

I disconnected the 5V power from the breadboard just to see what would happen to my toggle, increment, and decrement buttons. The main display went completely blank, which made sense, but I was really surprised to find out that the clock digits were actually still updating in the background when I pressed them. I realized this happens because I set up the buttons using the Arduino's internal pull-up resistors, so they don't even need the external 5V rail from the breadboard. Every button is wired straight to an Arduino digital pin and shares a common ground (GND) connection. Since that GND wire was still plugged in, pressing a button still pulled the pin LOW. It basically completes the circuit and reads the button press perfectly, even when the screen is completely dead.

The reference PDF specifies four 10 k ohm pull-down resistors for the buttons. I did not use these and instead used the Arduino's internal pull-up resistors, which reduced the number of components and made the breadboard less congested.

13.4 Visual Stutter During Editing

I noticed that the display briefly stutters whenever I press a button while editing the clock. I found that this happens because I used a delay(50) in the button-handling section to debounce the button. While the Arduino is waiting for those 50 ms, it stops running the multiplexing part of the code. This means the display that was active just before the button was pressed(hour tens display) stays on, while the other displays are not refreshed. As a result, one digit becomes noticeably brighter for a moment, causing a small flash or stutter. This helped me understand that although using a delay is a simple way to debounce buttons, it can also interrupt other parts of the program that need to keep running continuously.

13.5 Incorrect Hour Value When Decrementing

I found another issue while decrementing the hour digits. When the time is 09:00 and I decrement the tens digit of the hour, it rolls over to 2. This would temporarily produce 29:00, which is outside the valid range of the clock. The code detects this invalid value and forces the hour units digit to 0, resulting in 20:00. Although this prevents an invalid hour such as 29:00 from remaining on the clock, the result is not very intuitive because changing 09:00 in this way would be expected to move to the highest valid hour instead of jumping to 20:00. This showed me that preventing invalid values and giving the user the expected behavior are two different problems.

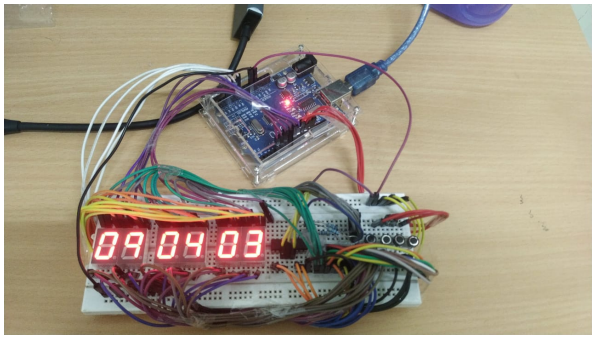


Figure 48: Before decrement of hour tens display

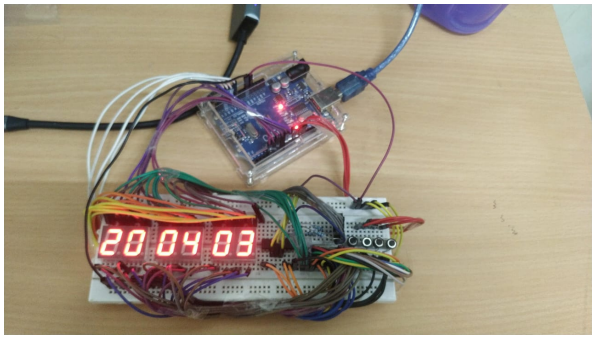
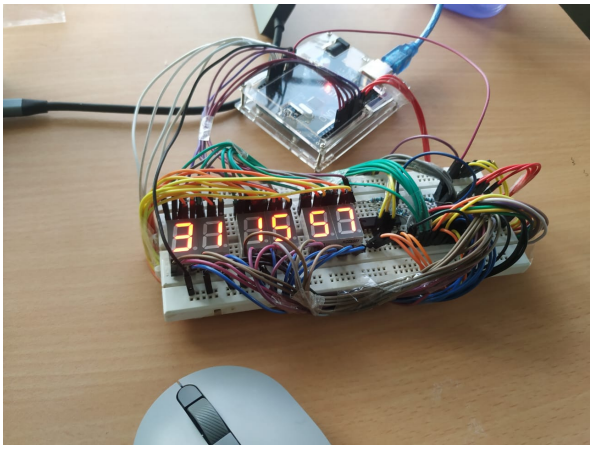


Figure 49: After decrement of hour tens display

13.6 Unexpected 31:15:57 Display During Editing

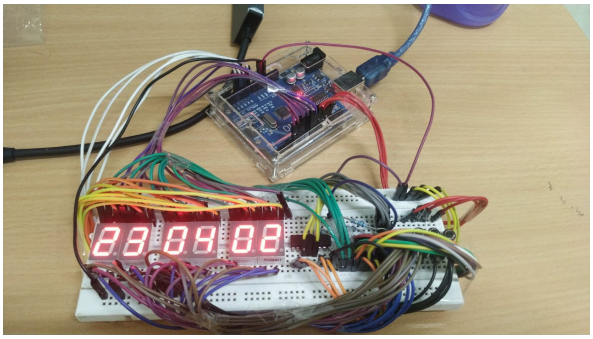
While editing the clock, I also encountered a strange situation where the display showed 31:15:57 and later returned to 20:16:00. At first, this looked like a software problem because 31:15:57 is not a valid time according to the clock's limits. However, after investigating the behaviour, I found that the clock's

internal value had actually remained at 20:15:57. The problem was caused by a loose physical connection on the A0 pin, which connects to BCD Bit A and then to pin 7 of the IC 7447. I concluded that the loose A0 connection was causing the A bit to be read as HIGH, changing the value that the 7447 received from the Arduino. This caused some even digits to appear as the next odd digit on the display, even though the Arduino's internal timekeeping was still correct. Once the physical connection settled and the A bit could be pulled LOW again, the display returned to showing the actual internal time, which was 20:16:00 by then.



At another point during testing, I noticed that every digit on the display was odd, including the seconds, which appeared to count as 1, 3, 5 and so on. This led me to investigate the same A0 connection more closely. Since A is the least significant bit of the BCD input, a loose connection that leaves this bit HIGH changes an even BCD value into the next odd value. For example, an internal value of 0 would be interpreted by the decoder as 1, 2 would appear as 3, and 4 would appear as 5. Odd numbers already have the A bit set, so they remain unchanged. This explained why the display appeared to stay on each odd number for two seconds at a time even though the Arduino was still counting correctly internally.

14 My working digital clock(without any improvements)



15 Improvements Made to the Code

After getting the clock working, I went back to the original code and looked at some of the problems I had noticed while testing it. Instead of changing the whole program, I tried to fix these issues while keeping the main logic of the clock the same.

15.1 Non-blocking button debouncing

In the original code, I used a delay(50) after pressing a button to deal with button bouncing. I later realised that this was causing a small problem because the Arduino could not do anything else during those 50 ms. This could make the display briefly stutter or flash when a button was pressed. I replaced this with a non-blocking debounce method that keeps track of the button's raw state, stable state and the time when the state last changed. The button press is only accepted if the signal remains stable for 50 ms. This allows the Arduino to keep refreshing the display and running the clock while the button is being debounced.

15.2 Explicit RUN and EDIT modes

I also changed the way I handled the clock's modes. Instead of using a paused variable, I introduced two clear modes: RUN and EDIT. I found this easier to understand because it makes it clear whether the clock is currently running normally or being edited. In EDIT mode, I can use the NEXT button to move between the six digits and use the increment and decrement buttons to change the selected digit. When the clock is in RUN mode, it goes back to counting normally.

15.3 Non-blocking multiplexing

The original display refresh used delay(1) while switching between the six displays. Although this worked, I realised that the delay was still making the Arduino stop briefly each time it refreshed a display. I changed this so that the multiplexing is controlled using millis(). The Arduino keeps track of the current display and switches to the next one when the required refresh time has passed. This means that the display can keep refreshing while the buttons and clock timing are also being handled.

15.4 Fixing the edit-mode timing problem

During testing, I noticed that the time could jump forward after I finished editing the clock. The reason was that millis() was still running in the background even though the clock itself was not supposed to be counting during edit mode. When I returned to RUN mode, the program was effectively including the time spent in edit mode. I fixed this by resetting the timing reference when leaving edit mode. This makes the clock start counting from the newly edited time instead of suddenly adding the time spent editing.

15.5 Long-press increment and decrement

I added another improvement to make setting the clock easier. If I press the increment or decrement button normally, the selected digit changes once. If I hold the button down, however, the value starts changing repeatedly after a short delay. This is useful when I need to change a digit by several numbers because I don't have to press the button separately for every number.

15.6 Keeping the original Boolean logic

Even though I made several changes to the code, I decided not to replace the original Boolean equations used for incrementing, decrementing and rolling over the digits. I wanted to improve the parts of the program that were causing problems without changing the main logic I had already developed. This also helped me keep the connection between the Boolean logic, the counting behaviour and the actual circuit. Most of my changes were therefore focused on making the button handling, display refresh and edit mode work more smoothly and reliably.

16 My final digital clock(with improvements)

After making these changes, I tested the clock again to check whether the problems I had identified were reduced. I checked the display while pressing the buttons, tested holding the increment and decrement buttons, entered and exited edit mode, and checked that the clock continued from the edited time. I

also checked that the display refresh continued while the buttons were being handled.

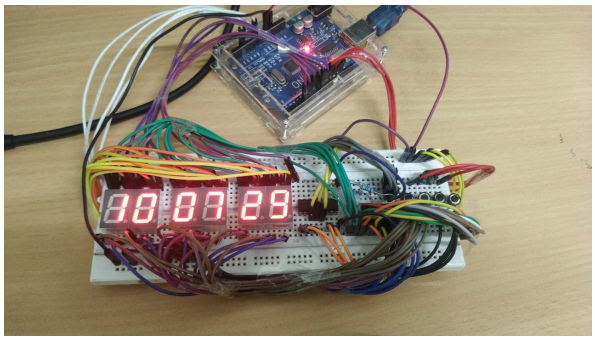


Figure 50: Clock working without any flickering

17 Overall learning and Development

17.1 Change in my understanding of how the clock works

I originally believed that the entire program ran on the Arduino and all that's left is simply connecting the wires to the LEDs. I soon came to know that there was more to it than what meets the eye. The digital clock is honestly just a massive brain dump of concepts, everything from BCD decoding and multiplexing to state logic and physical routing, all mashed into one build.

I spent a lot of time learning these concepts, researching the required learning resources and tried to test my knowledge on a few of the topics, they were very much dependent on critical thinking skills and a thorough grip on theoretical knowledge. I also realized just how tedious physically wiring the board can be, one small oversight and the whole project may stop working or worse, create invisible bugs. Running all the tests to check if the components allowed me to understand how the components work. I also learned how the use of multiplexing plays on the persistence of the human eye. I learned that the clock is not just the Arduino but multiple concepts and components working together.

17.2 Some of my major mistakes

- **Use of wrong resistors**

For most of the time I was using 10 kilo ohm resistor instead of the standard 220 ohm resistors. I never really questioned why my displays were dim and suddenly brightened up when I used other resistors. When the sudden brightening happened, I assumed that all of them were 220 ohm resistors and that some of them simply had a problem which caused some to produce a bright display and the others dim. I just assumed that it was supposed to be that way and that

I was wrong in thinking they were meant to be bright. I later realized that I was simply confused between 240 ohm resistors and 10 kilo ohm resistors. After getting the components from the core labs, I actually did check the internet for the colour coding of 220 ohm resistors but somehow I got misled into believing the 10 kilo ohm resistors were 220 ohm resistors.

- **Incorrect breadboard connection assumption**

I watched a video explaining the basics of breadboards from Paul McWhorter. From the video, I thought that the entire positive and negative rails were connected, but I later realized that I was wrong when I was testing the IC 7447 in the core labs and the person in charge explained to me that I was connecting the jumper cables incorrectly as the entire row does not share a connection but are divided down the middle.

- **Incorrectly fixed high connection**

When I was wiring the jumper cables, I taped the 3 cables connecting pins 3,4,5 of the IC 7447 to the 5V supply. I later connected this set of jumper cables and tried uploading the code, this caused the display to count from 4 to 7 and display some other digits incorrectly. I first checked the A0-A3 pins which handled the DCBA inputs. When this also did not work, I removed all the displays, had to remove some jumper cables I had grouped earlier and uploaded another code to display 0-9 digits on one seven segment display. This allowed me to discover that I connected the group of wires to pins 2,3,4 rather than 3,4,5 which led to pin 2 always receiving a high input and hence causing errors in the display.

17.3 My biggest discoveries

- **Timer/edit-mode problem**

I came into the edit mode issue when I was checking the functionality of the clock and realized that the edit mode was not working as intended. I later found that the issue was because millis() continued to run while in edit mode, hence causing the unexpected issues.

- **Breadboard connectivity**

One of my biggest discoveries while building the circuit was understanding how the power rails on the breadboard actually work. At first, I assumed that the rows marked positive and negative were connected from one end to the other, so I thought that supplying power at one end would automatically provide power to the whole rail. However, I later found out that the power rails are split into two sections in the middle, and the two sections are not connected to each other unless a jumper wire is used to connect them. This explained why some of the components in my circuit were not receiving power even though they seemed to be connected to the same power rail. Once I understood this, I was able to find and fix some of the connectivity problems in my circuit. It also made me more careful about checking whether different parts of the breadboard were actually connected to the same 5 V and GND supply.

- **Multiplexing**

While building the digital clock, I learned how multiplexing allows multiple seven-segment displays to be controlled using the same set of connections. At first, I thought that each display would need its own four BCD input lines connected to the 7447. However, I learned that the Arduino can switch between the six displays very quickly, turning on one display at a time and sending it the correct digit. What I found interesting was that this works because of the persistence of vision of the human eye. Since the displays are switched so quickly, our eyes cannot notice each individual display turning off before the next one turns on. Instead, the images blend together, making it look like all six displays are lit continuously at the same time. This helped me understand that multiplexing does not actually turn on all the displays at once, but switches between them quickly enough to create the appearance that they are all on continuously.

17.4 What I still want to improve

- Get a better understanding of FSMs
- Learn more about debouncing
- Get a deeper understanding of timers and interrupts